FLAC/WAV 8-bit Level-Occupancy Analyzer — Implementation Plan (v2)

> **For Hermes:** Use subagent-driven-development skill to implement this plan task-by-task.

> **v2 revision:** accepts `.flac` *and* `.wav`; sample rate, bit depth, and channel count are read from the **stream container metadata** (via `soundfile`/libsndfile), never from the filename. Supports PCM_16 and PCM_24 at $F_s \in \{44100, 48000, 88200, 96000, 176400, 192$ Hz. Rounding convention pinned to **round-half-to-even** to match the generator (`multitone/synthesize.py` uses `np.round`).

Goal: A Python program that reads a `.flac` or `.wav` file (metadata + PCM audio, channel by channel), plots the waveform, derives an 8-bit quantized version of the signal, counts how often each of the 256 quantized levels is encountered, and reports the estimated time / time-fraction spent on every level over the full duration.

Architecture: One importable module `multitone/analyze_multitone.py` with pure, unit-testable functions (loader, exact integer-domain 8-bit quantizer, occupancy counter, error/SQNR statistics, matplotlib plots) plus an argparse CLI. Tests follow the repo convention in root `tests/`.

Tech Stack: Python 3.13 (`.venv`), numpy 2.5.1, soundfile 0.14.0 (libsndfile FLAC/WAV decode), matplotlib 3.11.1, pytest 9.1.1. All confirmed installed in `.venv`.

Environment note (critical): always run project code with env -u PYTHONPATH .venv/Scripts/python … — the agent shell's `PYTHONPATH` points at the hermes-agent cp311 venv whose numpy 2.4.3 breaks numpy imports under the cp313 project venv.

Ground truth (measured — use as acceptance criteria)

Reference files (repo, generator-produced)

Property	24-bit FLAC (attached)	16-bit FLAC (sibling)
File	‘multitone/audio/ PrimeToneNewPhi[Nch 1] [Frac 03][LFL 043][00020-43200] [Nfr 031]-[96000-24]-[1s]-[-0.25dB]-[057923]-[8.7282]-[RD].flac‘	‘multitone/audio/ PrimeToneNewPhi[Nch 1] [Frac 03][LFL 043][00020-43200] [Nfr 031]-[96000-16]-[1s]-[-0.25dB]-[080734]-[8.6800]-[RD].flac‘
Stream	PCM_24, mono, 96 000 Hz, 96 000 frames, 1.000000 s	PCM_16, mono, 96 000 Hz, 96 000 frames, 1.000000 s
Sample range	±8 150 605 (24-bit)	±31 837 (16-bit)
Peak / RMS	−0.250 / −8.98 dBFS	−0.250 / −8.93 dBFS
8-bit levels occupied	**249/256, span [−124, +124]**	**249/256, span [−124, +124]**
Most occupied level	**−8: 816 samples** (8.500 ms, 0.850 %)	**−1: 882 samples** (9.188 ms, 0.919 %); level 0: 824
max $\|e\|$	3.90613 × 10^{-3} < D/2	**3.90625 × 10^{-3} = D/2 exactly** (ties present)
SQNR (8-bit)	43.96 dB	44.01 dB
Tie samples (m ≡ D/2 mod D)	**0**	**364** (182 negative) — convention matters

Both files carry **no in-stream text tags** (no Vorbis comment): the ”metadata” is the

stream header (codec, rate, channels, frames) — which is exactly what the program reads. The structured filename is a generator convention and is **not** used as a source of truth.

Loading semantics (verified by round-trip probes)

- `sf.read(path, dtype="int32", always_2d=True)` returns PCM_24 left-justified in s32 (low 8 bits zero — verified) and PCM_16 left-justified (low 16 bits zero — verified). True values: `x >> 8` (24-bit) / `x >> 16` (16-bit), arithmetic shift, exact, sign-preserving.
- WAV round-trips verified exact: synthesized PCM_16 (48 kHz stereo, 44.1 kHz stereo) and PCM_24 (88.2 kHz) WAVs written with `sf.write` and read back bit-exact under the scheme above.
- `sf.info` exposes `subtype` (PCM_16 / PCM_24), `samplerate`, `channels`, `frames`, `duration` — the authoritative metadata.

Generator rounding convention (verified in source)

`multitone/synthesize.py:119,126` — `RD` mode quantizes with `np.round`, i.e. **round-half-to-even** (banker's rounding, IEEE 754). The 16-bit file's 364 tie samples make the convention observable: half-even gives top level -1 with 882 samples; half-up would give 887. The plan implements half-even in the integer domain and cross-checks against `np.rint` in tests.

The mathematics

Audience: experts in signal processing / numerical analysis. Notation: N samples, $T_s = F_s^{-1}$, duration $T = NT_s$; $m[n] \in \{-2^{b-1}, \dots, 2^{b-1}-1\}$ the two's-complement sample at bit depth $b \in \{16, 24\}$; normalized $\hat{x}[n] = m[n]\,2^{-(b-1)} \in [-1, 1)$.

1. Stream model

The container supplies (b, F_s, C) — bit depth, rate, channel count — from the stream header, not the filename. For generator-produced files the signal is $x[n] = \sum_{k=1}^{K} a_k \cos(2\pi f_k n/F_s + \varphi_k)$, peak-normalized to -0.25 dBFS; the program does not need to know K or f_k (they are irrelevant to the occupancy analysis, which is amplitude-domain).

Left-justified s32 storage: libsndfile pads narrow PCM in a 32-bit container, so the loader's right shift by $32 - b$ recovers the true b-bit two's-complement value exactly. This is a pure bit operation — no rounding, no loss.

2. The 8-bit mid-tread uniform quantizer

256 reconstruction levels $\ell_k = kD$, $k \in \{-128, \dots, 127\}$, step $D = 2/256 = 2^{-7}$ (mid-tread: top level $127D$, bottom $-128D = -1$ — the standard two's-complement asymmetry). Decision regions $R_k = [(k - \frac{1}{2})D,\ (k + \frac{1}{2})D)$, last closed on the right.

In sample units the step is $D \cdot 2^{b-1} = 2^{b-8}$ ($= 2^8$ for 16-bit, 2^{16} for 24-bit), so the quantizer is **exact in the integer domain**. With $D_b = 2^{b-8}$:

$$q[n] = \mathrm{round}_{\mathrm{even}}\left(\frac{m[n]}{D_b}\right),$$

computed with floor division and a remainder test (no floating point):

```
k = m // D_b          (floor division; k = floor(m/D_b) for m < 0 too)
r = m − k·D_b         (0 ≤ r < D_b)
q = k + [ 2r > D_b  ∨  (2r = D_b ∧ k odd) ]
```

The bracketed term rounds the remainder up exactly when it exceeds half a step, or equals it *and* the candidate level k is odd — i.e. ties go to the **even** level. This matches `np.round`/`np.rint` bit-for-bit (verified) and is the IEEE 754 default, so it is the correct default even for files not produced by this generator.

Tie analysis. Ties occur at $m \equiv D_b/2 \pmod{D_b}$. The 24-bit reference file has 0 of them; the 16-bit reference file has 364 (182 negative), where the convention changes individual counts (e.g. $m = -21376$: half-even → −84, half-up → −83; $m = 640$: half-even → 2, half-up → 3). These are the test vectors in Task 2.

3. Quantization error and SQNR

$e[n] = \hat{x}[n] - q[n]D \in [-D/2,\, D/2]$; the bound is attained exactly at tie samples (16-bit file: $\max|e| = D/2 = 3.90625 \times 10^{-3}$) and approached within 2^{-23} otherwise (24-bit file: 3.90613×10^{-3}).

Classical result: if e is (approximately) uniform on $[-D/2, D/2]$ and uncorrelated with $\hat{x}$, then $\mathbb{E}[e^2] = D^2/12$, a noise floor $10\log_{10}(D^2/12) = -52.8$ dBFS. For a full-scale sine, $\sigma_x^2 = (2^7 D)^2/2$ gives the textbook SQNR $\approx 6.02N_b + 1.76 = 49.92$ dB at $N_b = 8$ ($6.02 = 10\log_{10} 4$ dB/bit; $+1.76 = 10\log_{10} 6$ dB).

Both reference files are 31-tone sums (CLT-approximately-Gaussian amplitude), RMS ≈ −9 dBFS, so SQNR $= 10\log_{10}(\sum_n \hat{x}^2 / \sum_n e^2) \approx -9 + 52.8 \approx 44$ dB — measured 43.96 / 44.01 dB, confirming the error model. (The 0.95-FS sine test in Task 4 checks the full-scale reference case: $10\log_{10} 6 + 20\log_{10}(0.95 \cdot 128) = 49.48$ dB.)

4. Level occupancy as a time estimate

$$N_k = \sum_{n=0}^{N-1} \mathbf{1}\{\mathcal{Q}(\hat{x}[n]) = k\}$$

via `np.bincount` (single $O(N)$ pass, exact integer counts). Each sample represents the interval $[nT_s, (n+1)T_s)$, so the **dwell-time** and **time-fraction** estimates are

$$\hat{t}_k = N_k\, T_s = \frac{N_k}{F_s}, \qquad \hat{p}_k = \frac{\hat{t}_k}{T} = \frac{N_k}{N}, \qquad \sum_k \hat{t}_k = T \text{ (exact).}$$

Interpretation: $\hat{p}_k$ is the plug-in estimator of $p_k = P(\mathcal{Q}(X) = k)$ for X a uniformly drawn sample of the signal. If samples were i.i.d., $N_k \sim \text{Binomial}(N, p_k)$ with 1σ error $\sqrt{p_k(1-p_k)/N}\,T$ — for the most occupied level of the 16-bit file ($\hat{p} = 9.19 \times 10^{-3}$, $N_k = 882$) that is ≈ 0.30 ms. For a deterministic multitone the samples are strongly correlated, so this binomial SE is only nominal; the real discretization error is §7.

5. The occupancy histogram as a pdf estimator

If the continuous amplitude has pdf f, the midpoint rule gives

$$p_k = \int_{(k-\frac{1}{2})D}^{(k+\frac{1}{2})D} f(u)\,du = D\,f(kD) + O\!\left(D^2 \max|f''|\right),$$

so $\hat{p}_k/D$ is a histogram pdf estimate at the level centres — the occupancy plot carries both readings (time primary, pdf estimate on the twin axis). Two checks on the reference files:

- **Gaussian cross-check:** the 31-tone sum is approximately Gaussian with $\sigma = \mathrm{rms}(\hat{x}) \approx 0.356$, so $p_0 \approx D/(\sigma\sqrt{2\pi}) \approx 0.88\%$ vs measured 0.84 % (24-bit) / 0.86 % (16-bit) — the few-percent gap is expected (a convolution of 31 arcsine laws, not exactly Gaussian).
- **Deterministic reachability:** both files occupy exactly $[-124, 124]$ because peak $0.9716 < (124 + \frac{1}{2})D = 0.9727$: the top 3 levels per side are unreachable — a consequence of the -0.25 dBFS normalization, not a statistic.

6. Numerical-analysis notes

- **Exactness.** The integer pipeline (shift, floor division, remainder test, `bincount`) involves **no rounding at all**; the only rounding in the program is the final division N_k/N (relative error $\lesssim 2^{-53}$, twelve orders below the binomial SE). $\hat{x} = m\,2^{-(b-1)}$ and e use power-of-two scaling, exact in binary floating point.
- **Overflow.** $m - kD_b$, $2r$ stay in $[0, 2D_b)$; with int64 there is no overflow path at $b \le 24$ (and even int32 would suffice, but int64 is free).
- **Complexity.** $O(ND)$ time, $O(256)$ extra memory; $N \le 384{,}000$ (384 kHz × 1 s) is trivial.
- **Determinism.** No RNG, no sorts; bit-reproducible outputs.
- **Testable invariants** (all asserted in the suite): $\sum_k N_k = N$; $\max|e| \le D/2$; $0 \le \hat{p}_k \le 1$; level span $\subseteq [-128, 127]$; integer quantizer ≡ `np.rint` on all reference data; loader round-trips exact.

7. Caveats

- **Sample resolution vs continuous dwell time.** $\hat{t}_k = N_k T_s$ counts intervals whose *sample value* falls in R_k. The true continuous dwell time of $x(t)$ in R_k differs by $O(T_s \max|\dot{x}|)$ per boundary crossing; with the top generator tone at 43.2 kHz (near Nyquist at 48 kHz) $\max|\dot{x}|$ is large, so the count is a consistent time-average of $\mathbf{1}\{Q(x) = k\}$, not an exact per-crossing dwell time. Continuous dwell (interpolated boundary crossings) is a noted extension, out of scope.
- **Tie convention for foreign files.** Half-even is the IEEE 754 / `np.round` default and matches this generator; a file produced by a different tool with a different tie break may differ on at most (number of ties) sample-counts at tie-adjacent levels.
- **Unsupported containers** (float PCM, μ-law/A-law, >24-bit, non-whitelisted F_s) are rejected with an explicit error, not silently coerced.

Program design

```
multitone/analyze_multitone.py      # the program (importable functions + main())
tests/test_analyze_multitone.py     # unit + integration tests
multitone/analysis_out/             # generated: waveform.png, occupancy_ch*.png,
    ↪occupancy_ch*.csv
```

Function	Contract
'load_audio(path) -> AudioSignal'	extension ∈ {.flac, .wav}; 'sf.info' → subtype ∈ {PCM_16, PCM_24} and F_s ∈ whitelist, else 'ValueError'; 'sf.read(dtype="int32", always_2d=True)'; 'raw = (x » (32-b)).astype(int64)'; per-channel access via 'sig.channel(c)'

‘quantize_8bit(m, sab) -> int8‘	$q = \text{round}_{even}(m/2^{sab-8}), exactint64(floor + remaindertest), clippedto[-128, 127]$
‘level_occupancy(q) -> (counts, levels)‘	‘np.bincount(q.astype(int64)+128, minlength=256)‘ → (int64[256], int8[256])
‘dwell_times(counts, sr) -> float64[256]‘	$N_k/F_s seconds$
‘quant_error_stats(m, q, sab) -> dict‘	‘max_abs_err‘, ‘half_step‘ (= 1/256), ‘sqnr_db‘
‘plot_waveform(sig, q_by_ch, out_dir) -> Path‘	per channel: full-duration row + zoom row with the 8-bit staircase overlay
‘plot_occupancy(counts, levels, sr, ch, out_dir) -> Path‘	256-bar time chart + twin-axis pdf estimate $\hat{p}_k/D$
‘main(argv=None) -> int‘	argparse CLI; console report per channel; full 256-row CSV per channel

CLI:

```
env -u PYTHONPATH .venv/Scripts/python -m multitone.analyze_multitone [PATH] [--out
    ↪DIR] [--no-plot]
```

(default `PATH` = the attached 24-bit FLAC; default `--out multitone/analysis_out/`)

”Channel by channel”: `raw` is `(frames, channels)`; the analysis loop, waveform rows, occupancy figures, and CSVs are all per-channel. Both reference files are mono; stereo files (e.g. the 48 kHz/44.1 kHz WAV round-trip tests) go through the same loop.

Tasks

Task 1: Container-aware loader (Fs/bits/channels from stream metadata)

Objective: read `.flac`/`.wav` with validation of subtype and rate, producing true bit-depth int64 samples per channel.

Files:

- Create: `multitone/analyze_multitone.py`
- Create: `tests/test_analyze_multitone.py`

Step 1: Write failing tests

```
from pathlib import Path

import numpy as np
import pytest
import soundfile as sf

from multitone.analyze_multitone import load_audio

FLAC24 = Path(
    "multitone/audio/PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200]"
    "[Nfr 031]-[96000-24]-[1s]-[-0.25dB]-[057923]-[ 8.7282]-[RD].flac")
FLAC16 = Path(
    "multitone/audio/PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200]"
    "[Nfr 031]-[96000-16]-[1s]-[-0.25dB]-[080734]-[ 8.6800]-[RD].flac")


def test_load_flac24_ground_truth():
    if not FLAC24.exists():
```

```
        pytest.skip("attached flac not present")
    sig = load_audio(FLAC24)
    assert (sig.samplerate, sig.frames, sig.channels, sig.bits) == (
        96000, 96000, 1, 24)
    m = sig.channel(0)
    assert m.dtype == np.int64 and m.shape == (96000,)
    assert m.min() == -8150605 and m.max() == 8150605
    assert abs(20 * np.log10(m.max() / 2**23) + 0.25) < 0.005  # -0.25 dBFS


def test_load_flac16_ground_truth():
    if not FLAC16.exists():
        pytest.skip("16-bit flac not present")
    sig = load_audio(FLAC16)
    assert (sig.samplerate, sig.frames, sig.channels, sig.bits) == (
        96000, 96000, 1, 16)
    m = sig.channel(0)
    assert m.min() == -31837 and m.max() == 31837
    assert abs(20 * np.log10(m.max() / 2**15) + 0.25) < 0.005


def test_wav_roundtrip_16_and_24_stereo(tmp_path):
    rng = np.random.default_rng(7)
    w16 = rng.integers(-2**15, 2**15, size=(44100, 2)).astype(np.int16)
    f16 = tmp_path / "s16.wav"
    sf.write(str(f16), w16, 44100, subtype="PCM_16")
    s16 = load_audio(f16)
    assert (s16.samplerate, s16.channels, s16.bits) == (44100, 2, 16)
    assert np.array_equal(s16.raw[:, 0], w16[:, 0].astype(np.int64))
    assert np.array_equal(s16.raw[:, 1], w16[:, 1].astype(np.int64))

    w24 = (rng.integers(-2**23, 2**23, size=(88200, 2)).astype(np.int32) << 8)
    f24 = tmp_path / "s24.wav"
    sf.write(str(f24), w24, 88200, subtype="PCM_24")
    s24 = load_audio(f24)
    assert (s24.samplerate, s24.channels, s24.bits) == (88200, 2, 24)
    assert np.array_equal(s24.raw, (w24 >> 8).astype(np.int64))


def test_rejects_bad_extension_and_rate_and_subtype(tmp_path):
    bad_ext = tmp_path / "x.aiff"
    bad_ext.write_bytes(b"RIFF....")
    with pytest.raises(ValueError, match="extension"):
        load_audio(bad_ext)

    w = np.zeros((3200, 1), dtype=np.int16)
    f32 = tmp_path / "badrate.wav"
    sf.write(str(f32), w, 32000, subtype="PCM_16")
    with pytest.raises(ValueError, match="sample rate"):
        load_audio(f32)

    fl = np.zeros((480, 1), dtype=np.float32)
    ffl = tmp_path / "float.wav"
    sf.write(str(ffl), fl, 48000, subtype="FLOAT")
    with pytest.raises(ValueError, match="subtype"):
        load_audio(ffl)
```

Step 2: Run tests to verify failure

Run: `env -u PYTHONPATH .venv/Scripts/python -m pytest tests/test_analyze_multitone.py -v` Expected: FAIL — `ModuleNotFoundError: multitone.analyze_multitone`

Step 3: Write minimal implementation — `multitone/analyze_multitone.py`:

```
"""Read a multitone .flac/.wav (stream metadata + PCM, channel by channel),
plot the waveform, and estimate the time spent on each 8-bit quantization
level. Sample rate / bit depth / channel count come from the stream header,
never from the filename.

Usage:
    env -u PYTHONPATH .venv/Scripts/python -m multitone.analyze_multitone [path]
"""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import numpy as np
import soundfile as sf

SUPPORTED_SUBTYPES = {"PCM_16": 16, "PCM_24": 24}
VALID_FS = (44100, 48000, 88200, 96000, 176400, 192000, 352800, 384000)


@dataclass(frozen=True)
class AudioSignal:
    path: Path
    samplerate: int
    channels: int
    frames: int
    duration: float
    subtype: str
    bits: int                       # true bit depth from the stream (16 or 24)
    raw: np.ndarray                 # int64, shape (frames, channels)

    def channel(self, c: int) -> np.ndarray:
        return self.raw[:, c]


def load_audio(path) -> AudioSignal:
    path = Path(path)
    if path.suffix.lower() not in (".flac", ".wav"):
        raise ValueError(
            f"unsupported extension {path.suffix!r} (need .flac or .wav)")
    info = sf.info(str(path))
    if info.subtype not in SUPPORTED_SUBTYPES:
        raise ValueError(
            f"unsupported subtype {info.subtype!r} (need PCM_16 or PCM_24)")
    if info.samplerate not in VALID_FS:
        raise ValueError(
            f"unsupported sample rate {info.samplerate} Hz "
            f"(need one of {VALID_FS})")
    bits = SUPPORTED_SUBTYPES[info.subtype]
    x, _ = sf.read(str(path), dtype="int32", always_2d=True)
    raw = (x >> (32 - bits)).astype(np.int64)   # left-justified s32 -> true value
    return AudioSignal(path=path, samplerate=info.samplerate,
                       channels=info.channels, frames=info.frames,
                       duration=float(info.duration), subtype=info.subtype,
                       bits=bits, raw=raw)
```

Step 4: Run tests to verify pass → 4 PASS **Step 5: Commit**

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): container-aware FLAC/WAV loader (stream metadata)"
```

Task 2: 8-bit quantizer (exact integer domain, round-half-to-even)

Objective: the mid-tread quantizer of §2 with the generator's rounding convention, incl. real tie vectors from the 16-bit file.

Files:

- Modify: `multitone/analyze_multitone.py`
- Modify: `tests/test_analyze_multitone.py`

Step 1: Write failing tests

```
from multitone.analyze_multitone import quantize_8bit


def test_quantize_boundaries():
    assert quantize_8bit(np.array([0]), 24)[0] == 0
    assert quantize_8bit(np.array([2**15 - 1]), 24)[0] == 0   # just below tie
    assert quantize_8bit(np.array([2**15]), 24)[0] == 0       # tie, even level 0
    assert quantize_8bit(np.array([3 * 2**15]), 24)[0] == 2   # tie, even level 2
    assert quantize_8bit(np.array([2**23 - 1]), 24)[0] == 127
    assert quantize_8bit(np.array([-2**23]), 24)[0] == -128


def test_quantize_ties_half_even_16bit():
    # real tie samples from the 16-bit reference file (D_b = 256)
    m = np.array([128, 640, -21376, -20352, -128], dtype=np.int64)
    q = quantize_8bit(m, 16)
    assert np.array_equal(q, [0, 2, -84, -80, 0])
    # half-up would give [1, 3, -83, -79, 0] — must NOT be the result
    q_up = np.floor(m.astype(np.float64) / 256.0 + 0.5)
    assert not np.array_equal(q, np.clip(q_up, -128, 127))


def test_quantizer_matches_np_rint():
    rng = np.random.default_rng(1)
    for sab in (16, 24):
        m = rng.integers(-2**(sab-1), 2**(sab-1), size=200_000, dtype=np.int64)
        q_ref = np.clip(np.rint(m / float(1 << (sab - 8))), -128, 127)
        assert np.array_equal(quantize_8bit(m, sab).astype(np.int64), q_ref)
```

Step 2: Run tests to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation

```
def quantize_8bit(m: np.ndarray, sab: int = 24) -> np.ndarray:
    """Map two's-complement samples to 8-bit mid-tread levels [-128, 127].

    q = round_even(m / 2^(sab-8)), computed in the integer domain (exact,
    round-half-to-even — matches the generator's np.round / IEEE 754),
    clipped to the 8-bit range (defensive).
    """
    if sab not in (16, 24):
        raise ValueError(f"unsupported bit depth: {sab}")
    m = np.ascontiguousarray(m, dtype=np.int64)
    d = 1 << (sab - 8)                  # step in sample units: 256 / 65536
    k = m // d                          # floor division (m may be negative)
    r = m - k * d                       # 0 <= r < d
    adj = (2 * r > d) | ((2 * r == d) & (k % 2 != 0))   # ties -> even level
    return np.clip(k + adj, -128, 127).astype(np.int8)
```

Step 4: Run tests to verify pass → 3 PASS **Step 5: Commit**

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
```

```
git commit -m "feat(multitone): exact int64 8-bit quantizer, round-half-to-even"
```

Task 3: Level occupancy and dwell times

Objective: the counts N_k, levels, and $\hat{t}_k = N_k / F_s$.

Files:

- Modify: `multitone/analyze_multitone.py`
- Modify: `tests/test_analyze_multitone.py`

Step 1: Write failing test

```
from multitone.analyze_multitone import dwell_times, level_occupancy


def test_occupancy_invariants():
    rng = np.random.default_rng(0)
    m = rng.integers(-2**23, 2**23, size=100_000, dtype=np.int64)
    q = quantize_8bit(m, 24)
    counts, levels = level_occupancy(q)
    assert counts.dtype == np.int64 and counts.shape == (256,)
    assert counts.sum() == m.size
    assert int(levels[0]) == -128 and int(levels[-1]) == 127
    t = dwell_times(counts, 96000)
    assert np.all(t >= 0)
    assert t.sum() == pytest.approx(m.size / 96000)
```

Step 2: Run test to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation

```
def level_occupancy(q: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """counts[k] = number of samples on level k-128; (int64[256], int8[256])."""
    counts = np.bincount(q.astype(np.int64) + 128, minlength=256)
    return counts, np.arange(-128, 128, dtype=np.int8)


def dwell_times(counts: np.ndarray, samplerate: int) -> np.ndarray:
    """t_k = N_k / F_s : estimated seconds spent on each level."""
    return counts.astype(np.float64) / float(samplerate)
```

Step 4: Run test to verify pass → PASS **Step 5: Commit**

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): level occupancy + dwell-time estimator"
```

Task 4: Quantization error statistics

Objective: $\max|e|$, the $D/2$ bound (attained at ties), and the SQNR — with a full-scale-sine reference and an exact-tie bound check.

Files:

- Modify: `multitone/analyze_multitone.py`
- Modify: `tests/test_analyze_multitone.py`

Step 1: Write failing test

```
from multitone.analyze_multitone import quant_error_stats
```

```python
def test_snr_095fs_sine_and_tie_bound():
    # 0.95 FS sine: no clipping, uniform-error model SQNR =
    # 10*log10(6) + 20*log10(0.95*128) = 49.48 dB
    n = 96000
    m = np.rint(0.95 * 2**23 * np.sin(2 * np.pi * 440 * np.arange(n) / 96000)
                ).astype(np.int64)
    q = quantize_8bit(m, 24)
    st = quant_error_stats(m, q, 24)
    assert st["max_abs_err"] <= st["half_step"]
    assert abs(st["sqnr_db"] - 49.48) < 1.0

    # exact tie (m = +D/2, 16-bit): |e| = D/2 attained
    mt = np.array([128, -128], dtype=np.int64)
    qt = quantize_8bit(mt, 16)
    stt = quant_error_stats(mt, qt, 16)
    assert stt["max_abs_err"] == stt["half_step"] == 1.0 / 256.0
```

Step 2: Run test to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation

```python
def quant_error_stats(m: np.ndarray, q: np.ndarray, sab: int = 24) -> dict:
    scale = float(1 << (sab - 1))
    x = m.astype(np.float64) / scale               # exact (power-of-two scaling)
    e = x - q.astype(np.float64) / 128.0           # level * D, D = 2^-7
    return {
        "max_abs_err": float(np.abs(e).max()),
        "half_step": 1.0 / 256.0,                  # D/2, D = 2^-7
        "sqnr_db": float(10.0 * np.log10(np.sum(x * x) / np.sum(e * e))),
    }
```

Step 4: Run test to verify pass → PASS **Step 5: Commit**

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): quantization error / SQNR statistics"
```

Task 5: Plots (waveform + occupancy)

Objective: the two figures — full/zoom waveform with the 8-bit staircase overlay, and the 256-level occupancy chart with the pdf-estimate twin axis.

Files:

- Modify: `multitone/analyze_multitone.py`
- Modify: `tests/test_analyze_multitone.py`

Step 1: Write failing test

```python
from multitone.analyze_multitone import plot_occupancy, plot_waveform


def test_plots_created(tmp_path):
    if not FLAC24.exists():
        pytest.skip("attached flac not present")
    sig = load_audio(FLAC24)
    m = sig.channel(0)
    q = quantize_8bit(m, sig.bits)
    counts, levels = level_occupancy(q)
    p1 = plot_waveform(sig, {0: q}, tmp_path)
    p2 = plot_occupancy(counts, levels, sig.samplerate, 0, tmp_path)
    assert p1.exists() and p1.stat().st_size > 10_000
    assert p2.exists() and p2.stat().st_size > 10_000
```

Step 2: Run test to verify failure → FAIL (ImportError)

Step 3: Write minimal implementation (append to `multitone/analyze_multitone.py`)

```
def _plt():
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    return plt


def plot_waveform(sig: AudioSignal, q_by_channel: dict, out_dir) -> Path:
    plt = _plt()
    t = np.arange(sig.frames) / sig.samplerate
    scale = 2.0 ** (sig.bits - 1)
    D = 2.0 / 256.0
    nzm = min(384, sig.frames)                    # zoom window (4 ms @ 96 kHz)
    zoom_ms = nzm / sig.samplerate * 1e3
    fig, axes = plt.subplots(2 * sig.channels, 1,
                             figsize=(11, 2.2 * sig.channels), sharex=True)
    axes = np.atleast_1d(axes)
    for c in range(sig.channels):
        full = sig.channel(c) / scale
        axes[2 * c].plot(t, full, lw=0.3, color="navy")
        axes[2 * c].set_ylabel(f"ch{c} full")
        axes[2 * c].set_ylim(-1.05, 1.05)
        axes[2 * c + 1].plot(t[:nzm], full[:nzm], lw=0.4,
                             color="navy", label="original")
        axes[2 * c + 1].step(t[:nzm], q_by_channel[c][:nzm] * D,
                             where="mid", lw=0.9, color="crimson",
                             label="8-bit quantized")
        axes[2 * c + 1].set_ylabel(f"ch{c} {zoom_ms:.1f} ms zoom")
        axes[2 * c + 1].legend(loc="upper right", fontsize=8)
    axes[-1].set_xlabel("time (s)")
    fig.suptitle(f"{sig.path.name} — {sig.samplerate} Hz, "
                 f"{sig.frames} fr, {sig.channels} ch, {sig.bits}-bit")
    fig.tight_layout(rect=(0, 0, 1, 0.98))
    out = Path(out_dir) / "waveform.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    return out


def plot_occupancy(counts: np.ndarray, levels: np.ndarray, samplerate: int,
                   channel: int, out_dir) -> Path:
    plt = _plt()
    t = dwell_times(counts, samplerate)
    p = counts / counts.sum()
    fig, ax = plt.subplots(figsize=(11, 4))
    ax.bar(levels, t, width=1.0, color="steelblue")
    ax.set_xlabel("8-bit level $k$")
    ax.set_ylabel("time $t_k$ (s)")
    ax.set_title(f"Level occupancy, ch{channel}: {int(counts.sum())} samples, "
                 f"{int(np.count_nonzero(counts))}/256 levels occupied")
    ax2 = ax.twinx()
    ax2.plot(levels, p * 128.0, "r.", ms=3,
             label=r"$\hat p_k / D$ (pdf estimate)")
    ax2.set_ylabel(r"$\hat p_k / D$ (pdf estimate)")
    ax2.legend(loc="upper right", fontsize=8)
    ax.grid(alpha=0.3)
    fig.tight_layout()
    out = Path(out_dir) / f"occupancy_ch{channel}.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
```

```
    return out
```

Step 4: Run test to verify pass → PASS **Step 5: Commit**

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): waveform + occupancy plots"
```

Task 6: CLI, console report, end-to-end on both reference files

Objective: `main()` — per-channel report, top-10 table, full 256-row CSV, plots; e2e assertions pinned to the ground-truth table for **both** the 24-bit and 16-bit files.

Files:

- Modify: `multitone/analyze_multitone.py`
- Modify: `tests/test_analyze_multitone.py`

Step 1: Write failing tests

```
from multitone.analyze_multitone import main


def test_cli_flac24(tmp_path, capsys):
    if not FLAC24.exists():
        pytest.skip("attached flac not present")
    rc = main([str(FLAC24), "--out", str(tmp_path), "--no-plot"])
    out = capsys.readouterr().out
    assert rc == 0
    assert "96000 Hz" in out and "PCM_24" in out
    assert "249/256" in out and "[-124, 124]" in out
    assert "43.96" in out                       # SQNR
    assert "  816" in out                       # top level -8
    csv = tmp_path / "occupancy_ch0.csv"
    assert len(csv.read_text().strip().splitlines()) == 257


def test_cli_flac16(tmp_path, capsys):
    if not FLAC16.exists():
        pytest.skip("16-bit flac not present")
    rc = main([str(FLAC16), "--out", str(tmp_path), "--no-plot"])
    out = capsys.readouterr().out
    assert rc == 0
    assert "PCM_16" in out
    assert "249/256" in out and "[-124, 124]" in out
    assert "44.01" in out                       # SQNR
    assert "  882" in out                       # top level -1 (half-even)
```

Step 2: Run tests to verify failure → FAIL (ImportError: `main`)

Step 3: Write minimal implementation (append to `multitone/analyze_multitone.py`)

```
import argparse

DEFAULT_PATH = Path(__file__).resolve().parent / "audio" / (
    "PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200]"
    "[Nfr 031]-[96000-24]-[1s]-[-0.25dB]-[057923]-[ 8.7282]-[RD].flac")


def main(argv=None) -> int:
    ap = argparse.ArgumentParser(
        description="FLAC/WAV 8-bit level-occupancy analyzer "
                    "(Fs/bits/channels from stream metadata)")
    ap.add_argument("path", nargs="?", default=str(DEFAULT_PATH))
```

```
    ap.add_argument("--out",
                    default=str(Path(__file__).resolve().parent / "analysis_out"))
    ap.add_argument("--no-plot", action="store_true")
    a = ap.parse_args(argv)

    sig = load_audio(Path(a.path))
    out_dir = Path(a.out)
    out_dir.mkdir(parents=True, exist_ok=True)

    print(f"file: {sig.path.name}")
    print(f"  {sig.samplerate} Hz, {sig.frames} frames, {sig.channels} ch, "
          f"{sig.subtype}, {sig.duration:.6f} s")

    q_all = {}
    for c in range(sig.channels):
        m = sig.channel(c)
        q = quantize_8bit(m, sig.bits)
        q_all[c] = q
        counts, levels = level_occupancy(q)
        st = quant_error_stats(m, q, sig.bits)
        scale = float(2.0 ** (sig.bits - 1))
        peak = m.max() / scale
        rms = float(np.sqrt(np.mean((m / scale) ** 2)))
        occ = np.nonzero(counts)[0]
        print(f"channel {c}: peak {20*np.log10(peak):+.3f} dBFS, "
              f"rms {20*np.log10(rms):+.2f} dBFS, "
              f"levels {occ.size}/256 [{int(levels[occ[0]])}, "
              f"{int(levels[occ[-1]])}], SQNR {st['sqnr_db']:.2f} dB, "
              f"max|e| {st['max_abs_err']:.9f} (<= {st['half_step']:.9f})")
        top = sorted(np.argsort(counts)[::-1][:10])
        print("  top-10 levels (level, samples, ms, %):")
        for k in top:
            print(f"    {int(levels[k]):+4d}  {counts[k]:6d}  "
                  f"{dwell_times(counts, sig.samplerate)[k]*1e3:8.3f}  "
                  f"{counts[k]/m.size*100:6.3f}")
        np.savetxt(out_dir / f"occupancy_ch{c}.csv",
                   np.column_stack([levels, counts,
                                    dwell_times(counts, sig.samplerate)]),
                   delimiter=",", header="level,samples,seconds", comments="")
    if not a.no_plot:
        print(f"plot: {plot_waveform(sig, q_all, out_dir)}")
        for c in range(sig.channels):
            counts, levels = level_occupancy(q_all[c])
            print(f"plot: {plot_occupancy(counts, levels, sig.samplerate, c,
    ↪out_dir)}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Step 4: Run the full suite

Run: `env -u PYTHONPATH .venv/Scripts/python -m pytest tests/test_analyze_multitone.py ↪-v` Expected: 11 passed, 0 skipped (both reference FLACs present)

Step 5: Run the real CLI on both files and inspect outputs

```
env -u PYTHONPATH .venv/Scripts/python -m multitone.analyze_multitone
env -u PYTHONPATH .venv/Scripts/python -m multitone.analyze_multitone \
  "multitone/audio/PrimeToneNewPhi[Nch 1][Frac 03][LFL 043][00020-43200][Nfr
    ↪031]-[96000-16]-[1s]-[-0.25dB]-[080734]-[ 8.6800]-[RD].flac"
```

Expected console output (must match the ground-truth table):

- 24-bit: `96000 Hz, 96000 frames, 1 ch, PCM_24; peak -0.250 dBFS; rms -8.98 dBFS; levels 249/256 [-124, 124]; SQNR 43.96 dB; max|e| 0.003906130 (<= 0.003906250); top -8 816 8.500 0.850.`
- 16-bit: `PCM_16; peak -0.250 dBFS; rms -8.93 dBFS; levels 249/256 [-124, 124]; SQNR 44.01 dB; max|e| 0.003906250 (<= 0.003906250)` (tie bound attained); `top -1 882 9.188 0.919.`

Then visually check `multitone/analysis_out/waveform.png` (full trace + zoom showing the staircase hugging the waveform) and `occupancy_ch0.png` (256 bars, mass concentrated near the centre, empty top/bottom 3 levels per side).

Step 6: Commit

```
git add multitone/analyze_multitone.py tests/test_analyze_multitone.py
git commit -m "feat(multitone): CLI + e2e ground-truth on 16/24-bit reference files"
```

Acceptance criteria

- `env -u PYTHONPATH .venv/Scripts/python -m pytest tests/test_analyze_multitone.py -v` → all green (11 tests).
- CLI on the 24-bit FLAC prints the ground-truth values (table above) and writes `multitone/analysis_out/` with `waveform.png`, `occupancy_ch0.png`, `occupancy_ch0.csv` (257 lines); same for the 16-bit FLAC.
- $\sum_k N_k = N$ exact and $\sum_k \hat{t}_k = T$ for both files.
- No filename parsing anywhere in the program: F_s, b, C come from the stream header; a mislabeled file still analyzes correctly.
- Program is generic in channel count and rate: stereo WAV round-trips (44.1/88.2 kHz) pass; the 352.8/384 kHz whitelist members go through the identical code path.

Risks, tradeoffs, open questions

- **Tie convention.** Pinned to round-half-to-even (IEEE 754 / `np.round`), matching this generator. For the 16-bit reference file the choice is observable (364 ties; top level 882 vs 887 under half-up) — Task 2's tie vectors guard it.
- **Unsupported subtypes/rates** (float PCM, μ-law/A-law, 32-bit int, non-whitelisted F_s, other extensions) are rejected with explicit `ValueError`s rather than coerced — a mislabeled or exotic file fails loudly.
- **352 800 / 384 000 Hz** have no repo file to pin ground truth; the whitelist + identical code path covers them (the WAV round-trip tests exercise two other whitelist rates).
- **Filename semantics** (`Frac`, `LFL`, `score`, `RD` tag) are deliberately ignored — the stream header is authoritative per the revised scope. If a future task wants name/stream cross-checks, add an explicit `--check-name` flag rather than implicit parsing.
- **Out of scope (noted extensions):** continuous (interpolated) dwell times, spectral analysis, stereo cross-channel comparison.